

Operations Runbook

Deployment, day-to-day operations and incident response

Document ID	OBE-RUN-001
Version	1.1
Date	2026-10-10
Status	Approved for the dev environment
Owner	khaipd18 (DevOps / Cloud)
Audience	Engineers who deploy, operate or support the platform
Classification	Internal
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Document control

Revision history

Version	Date	Change	Author
1.0	2026-10-09	First issue: access, bootstrap, routine procedures, monitoring, incident playbooks, teardown.	khaipd18
1.1	2026-10-10	First run on the new account: state bucket renamed, control plane log group managed by Terraform (74 resources), apply and destroy verified.	khaipd18

Related documents

- OBE-TDD-001 Technical Design Document (docs/manuals/technical-design.en.pdf)
- README.md: project summary and local test guide with kind

Contents

1	About this runbook	3
1.1	Purpose and use	3
1.2	Environment at a glance	3
1.3	Severity levels	3
2	Access and tools	4
2.1	Workstation tools	4
2.2	Connect to AWS and the cluster	4
2.3	Open the consoles	4
3	One-off procedures	5
3.1	RB-01 Bootstrap a new environment	5
3.2	RB-02 Tear down the environment	6
3.3	RB-03 Move to another AWS account	6
4	Routine procedures	7
4.1	RB-04 Release a new version of a service	7
4.2	RB-05 Roll back a service	7
4.3	RB-06 Change service configuration	7
4.4	RB-07 Add a new service	7
4.5	RB-08 Change infrastructure	8
4.6	RB-09 Scale	8
4.7	RB-10 Upgrade Kubernetes and add-ons	8
4.8	RB-11 Accept or fix a Checkov finding	9
4.9	RB-12 Handle image vulnerabilities	9
4.10	RB-13 Control cost	9
5	Monitoring	10
5.1	Daily checks	10
5.2	What to watch in Grafana	10
5.3	Logs	10
6	Incident playbooks	11
6.1	IR-01 CI fails at "Configure AWS credentials"	11
6.2	IR-02 Terraform state lock not released	11
6.3	IR-03 Terraform Apply does not start	11
6.4	IR-04 GitOps write-back rejected	11
6.5	IR-05 Argo CD application OutOfSync or Degraded	12
6.6	IR-06 Pod in ImagePullBackOff	12
6.7	IR-07 Pod in CrashLoopBackOff or OOMKilled	12
6.8	IR-08 Pod rejected by Pod Security Admission	12
6.9	IR-09 Calls between services time out	13
6.10	IR-10 Shop not reachable from the internet	13
6.11	IR-11 Pods Pending or node NotReady	13
6.12	IR-12 HPA shows <unknown> metrics	13
6.13	IR-13 Node drain blocked	13
6.14	IR-14 Shopping carts emptied	14
7	Disaster recovery	15
	Appendix A. Service and port reference	16
	Appendix B. Command reference	17
	Appendix C. Versions	18

1 About this runbook

1.1 Purpose and use

This runbook tells an engineer how to deploy and operate the Online Boutique platform: one-off procedures (bootstrap, teardown, account move), routine changes (release, rollback, configuration, infrastructure) and what to do when something breaks. The design behind it is described in the Technical Design Document (OBE-TDD-001).

Each procedure lists its prerequisites, numbered steps and a verification step. Run the steps in order. Commands assume a Bash shell at the repository root.

IMPORTANT

Git is the source of truth. Argo CD reverts any manual change made with `kubectl` in namespace `dev-eks` (self-heal). Apart from the diagnostic commands in this runbook, change the cluster through a pull request.

1.2 Environment at a glance

Item	Value
Region	ap-southeast-1
EKS cluster	khaipd18-eks-cluster (Kubernetes 1.35)
Application namespace	dev-eks (releases <service>-dev)
Other namespaces	argocd (Argo CD), monitoring (kube-prometheus-stack)
Public entry point	Service frontend-external-dev (Classic Load Balancer, port 80)
Terraform state	S3 khaipd18-obe-tf-state-445817183958, key dev/terraform.tfstate lock table khaipd18-devops-project-terraform-state-lock
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline
GitHub controls	Ruleset protect-main; environment production (reviewer khaipd18)

1.3 Severity levels

Level	Definition	Examples	Response
SEV1	Shop unavailable or checkout broken for all users	frontend down, load balancer gone, all pods failing	Start at once; fix or roll back first, investigate after
SEV2	A feature degraded, or delivery blocked	One service crash-looping, CI cannot push images, Argo CD stuck	Same working day
SEV3	No user impact	Warning-level scan findings, a single pod restart, documentation drift	Plan into normal work

The platform has a single owner (khaipd18). Record every SEV1/SEV2 incident as a GitHub issue with the timeline, cause and follow-up actions.

2 Access and tools

2.1 Workstation tools

Tool	Used for
AWS CLI v2	Credentials, kubeconfig, state backend, AWS checks
Terraform 1.14.8	Infrastructure (first apply and break-glass only; normally CI)
kubectl, Helm	Inspecting the cluster; installing Argo CD
GitHub CLI (gh)	Workflow runs, logs, approvals, repository settings
Typst	Rebuilding these PDF documents (docs/manuals/build.sh)

2.2 Connect to AWS and the cluster

1. Confirm the AWS identity and account:

```
aws sts get-caller-identity
```

2. Write the kubeconfig entry:

```
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl get nodes
```

NOTE

The cluster sets no `access_config`, so the API default applies: only the IAM principal that created the cluster has admin access at first [AWS]. Check the mode with `aws eks describe-cluster --name khaipd18-eks-cluster --query cluster.accessConfig`. To give another engineer access, switch the cluster to `API_AND_CONFIG_MAP` and create an access entry with an access policy [AWS] do it in Terraform so it is not lost on the next apply.

2.3 Open the consoles

All consoles are reached through `kubectl port-forward`; none of them is exposed to the internet.

```
# Argo CD: https://localhost:8080, user admin
kubectl -n argocd port-forward svc/argocd-server 8080:443
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo

# Grafana: http://localhost:3000, user admin
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
kubectl -n monitoring get secret kube-prometheus-stack-grafana \
  -o jsonpath='{.data.admin-password}' | base64 -d; echo

# Prometheus: http://localhost:9090
kubectl -n monitoring port-forward svc/kube-prometheus-stack-prometheus 9090:9090

# Shop URL
kubectl -n dev-eks get svc frontend-external-dev \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'; echo
```

Change the Argo CD admin password after the first login and delete `argocd-initial-admin-secret`.

3 One-off procedures

3.1 RB-01 Bootstrap a new environment

When: first deployment in an AWS account, or rebuilding after a teardown. **Duration:** about 45 minutes (EKS creation takes 15–20). **Cost:** hourly charges start at step 4 (see RB-13).

1. **Set the target account in GitHub.** Settings → Secrets and variables → Actions → Variables → AWS_ACCOUNT_ID. Or:

```
gh variable set AWS_ACCOUNT_ID --body <account-id>
```

2. **Check name clashes** if the account is shared: an account can have only one OIDC provider for token.actions.githubusercontent.com, and IAM role and policy names are unique per account (github-actions-*, GitHubActions-*).
3. **Create the state backend** (once per account). Bucket names are global; if you change them, also change terraform/backend.tf and the tf_state_bucket / tf_state_lock_table variables.

```
aws s3api create-bucket --bucket <state-bucket> --region ap-southeast-1 \
  --create-bucket-configuration LocationConstraint=ap-southeast-1
aws s3api put-bucket-versioning --bucket <state-bucket> \
  --versioning-configuration Status=Enabled
aws dynamodb create-table --table-name <lock-table> --region ap-southeast-1 \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH --billing-mode PAY_PER_REQUEST
```

4. **First apply from the workstation.** The GitHub roles do not exist yet, so CI cannot do it. The identity used here becomes the cluster admin.

```
cd terraform
aws sts get-caller-identity          # must be the target account
terraform init
terraform plan -out tfplan          # expect 74 resources to add
terraform apply tfplan
```

5. **Install Argo CD and the applications:**

```
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/
stable/manifests/install.yaml
kubectl apply -f gitops/argocd/namespaces.yaml
kubectl apply -f gitops/argocd/applicationset.yaml
kubectl apply -f gitops/argocd/monitoring.yaml --server-side
```

6. **Build the images.** Run the 5 CI workflows once (Actions → CI for ... Services → Run workflow), or:

```
for w in go dotnet java node py; do gh workflow run $w-services-ci.yaml --ref main;
done
```

CI pushes the images and writes the new tags into gitops/dev-eks/.

WARNING

Known issue: the ruleset protect-main rejects the CI bot's push of the new image tags (GitHub error GH013), so step 6 fails at "Update GitOps and Push Back". Until the roadmap fix is in place, disable the ruleset for the bootstrap and enable it again right after:

```
gh api -X PUT repos/khaipd18/online-boutique-gitops-pipeline/rulesets/24705847 -f
enforcement=disabled
# ... run step 6 and wait for all runs to finish ...
gh api -X PUT repos/khaipd18/online-boutique-gitops-pipeline/rulesets/24705847 -f
enforcement=active
```

Verify:

```
kubectl -n argocd get applications          # all Synced / Healthy
kubectl -n dev-eks get pods                # all Running, frontend has 2 replicas
kubectl -n dev-eks get hpa,pdb             # frontend HPA reads a CPU value
```

Open the shop URL and place an order; the confirmation page must appear.

3.2 RB-02 Tear down the environment

When: the environment is not needed. EKS has no free tier; the control plane, EC2 nodes, the NAT Gateway and interface endpoints are billed per hour [\[AWS\]](#).

1. Delete the applications and the load balancer first. The load balancer was created by Kubernetes and is not in the Terraform state; if it remains, terraform destroy fails on the VPC.

```
kubectl delete -f gitops/argocd/applicationset.yaml
kubectl -n dev-eks delete svc frontend-external-dev --ignore-not-found
```

2. Destroy the infrastructure (from the workstation, with admin credentials):

```
cd terraform && terraform destroy
```

3. Optional: delete the state bucket and lock table if the account will not be used again.
4. Optional: unset the GitHub variable so CI skips AWS steps again: `gh variable delete AWS_ACCOUNT_ID`.

Verify: `aws eks list-clusters --region ap-southeast-1` does not list the cluster, and the EC2 console shows no load balancer or NAT Gateway left in the region.

3.3 RB-03 Move to another AWS account

1. Run RB-02 in the old account (or accept that it keeps running).
2. Run RB-01 in the new account. No code change is needed: Terraform reads the account from the credentials and CI from `AWS_ACCOUNT_ID`.
3. The ECR addresses in `gitops/dev-eks/values-*.yaml` still point to the old account until CI rewrites them in RB-01 step 6.

4 Routine procedures

4.1 RB-04 Release a new version of a service

1. Merge the change under `src/<service>/` into `main`.
2. The language workflow lints, tests, builds, scans (Trivy) and pushes the image tagged with the commit SHA, then commits the new tag to `gitops/dev-eks/values-<service>.yaml` with `[skip ci]`.
3. Argo CD detects the commit (default polling every 3 minutes) and rolls out the new image.

Verify:

```
gh run list --limit 5
kubectl -n argocd get application <service>-dev
kubectl -n dev-eks rollout status deploy/<service>-dev
kubectl -n dev-eks get deploy <service>-dev -o jsonpath='{..image}'; echo
```

4.2 RB-05 Roll back a service

Every image tag is a commit SHA and tags are immutable, so a rollback means pointing the values file back at the previous tag.

1. Find the commit that changed the tag:

```
git log --oneline -- gitops/dev-eks/values-<service>.yaml
```

2. Revert it in a pull request (or as an admin push in an emergency):

```
git revert <commit-of-bad-tag>
```

3. Argo CD rolls back to the previous image. Follow with `kubectl -n dev-eks rollout status deploy/<service>-dev`.

WARNING

Do not use `kubectl set image` or `kubectl rollout undo`: Argo CD self-heal restores the tag from Git within minutes.

4.3 RB-06 Change service configuration

Environment variables, resources, probes, replicas, autoscaling and NetworkPolicy callers are set in `gitops/dev-eks/values-<service>.yaml`.

1. Edit the values file in a branch and render it locally to check:

```
helm template <service>-dev helm-charts -f gitops/dev-eks/values-<service>.yaml -n dev-eks
```

2. Open a pull request; the Security Scan workflow runs Checkov on the rendered manifests.
3. Merge; Argo CD applies the change.

TIP

When a service starts calling another one, add the caller's release name to the callee's `networkPolicy.allowFrom`, otherwise the call times out (see IR-09).

4.4 RB-07 Add a new service

1. Add `gitops/dev-eks/values-<name>.yaml` (copy a similar service; keep the chart's security defaults).

2. Add `- name: <name>` to the list generator in `gitops/argocd/applicationset.yaml`.
3. Add the service to the `allowFrom` lists of the services it calls, and list its callers in its own `allowFrom`.
4. For a new image: add an ECR repository name to `repository_names` in `terraform/variables.tf` and the service to the matching CI workflow (paths filter and dispatch list).
5. Apply the `ApplicationSet` change once: `kubectl apply -f gitops/argocd/applicationset.yaml` (the `ApplicationSet` itself is not managed by Argo CD).

4.5 RB-08 Change infrastructure

1. Change `terraform/` in a branch and open a pull request.
2. **Checkov Scan** must pass. **Terraform Plan** runs with the read-only role; read the plan in the job log.
3. Get the pull request reviewed, then merge.
4. The **Terraform Apply** job waits for approval. Approve it in Actions → the run → Review deployments → production → Approve and deploy, or:

```
gh run list --workflow terraform.yaml --limit 1
gh run view <run-id> # shows "waiting for review"
```

5. Check the apply log, then verify the resource in AWS.

IMPORTANT

Approve only a plan you have read. The apply job runs `terraform apply -auto-approve` with `AdministratorAccess` once it is approved.

4.6 RB-09 Scale

- **frontend:** the HPA keeps 2–4 replicas at 70 % CPU. Change the range in `values-frontend.yaml` (`autoscaling.minReplicas / maxReplicas`).
- **Other services:** set `replicaCount` or turn on `autoscaling.enabled` in their values file. A `PodDisruptionBudget` is added automatically from 2 replicas. Keep `redis-cart` at 1 (data in `emptyDir`).
- **Nodes:** change `eks_node_group_scaling_config` (min/desired/max, now 1/2/3) through RB-08. There is no cluster autoscaler: `desired_size` is the number of nodes.

4.7 RB-10 Upgrade Kubernetes and add-ons

Kubernetes 1.35 leaves standard support on 27 March 2027 [\[AWS\]](#). Upgrade one minor version at a time, in a pull request (RB-08):

1. Read the EKS release notes for the target version and check deprecated APIs in the manifests.
2. Bump `eks_k8s_version` and apply: this upgrades the control plane only.
3. Upgrade the managed node group. The Terraform module sets no node group version, so the nodes keep their version until updated, and they cannot be newer than the control plane [\[AWS\]](#):

```
aws eks update-nodegroup-version --cluster-name khaipd18-eks-cluster \
  --nodegroup-name khaipd18-eks-cluster-node-group --kubernetes-version <version>
```

4. Find add-on versions for the new Kubernetes version and bump the `eks_*_version` variables:

```
aws eks describe-addon-versions --addon-name vpc-cni --kubernetes-version <version> \
  --query 'addons[0].addonVersions[0:3].
[addonVersion,compatibilities[0].defaultVersion]' --output text
```

Repeat for `coredns`, `kube-proxy` and `metrics-server`.

Verify: `kubectl get nodes` shows the new version on every node; all Argo CD applications are Healthy; the shop checkout works.

4.8 RB-11 Accept or fix a Checkov finding

1. Prefer fixing the Terraform. If the finding is acceptable, write down why (a comment next to the resource).
2. Regenerate the baseline in a pull request so it gets reviewed:

```
checkov --config-file .checkov.yaml -d terraform --framework terraform --create-baseline
```

4.9 RB-12 Handle image vulnerabilities

1. Open the GitHub **Security** tab → Code scanning, filter by tool **Trivy** and the image (trivy-`<service>`). Each run also writes a count per image in the job summary.
2. Findings come from base images and dependencies in `src/`, which this repository does not change. Upgrading them is a code change for the application owners; this repository then switches the gate to blocking by setting `blocking: 'true'` in the Trivy step.
3. The SBOM (CycloneDX) for each image is kept as a run artifact for 30 days (sbom-`<service>`).

4.10 RB-13 Control cost

Billable item	Notes
EKS control plane	Per hour while the cluster exists
EC2 nodes	2 × t3.medium on-demand by default
NAT Gateway	Per hour and per GB processed [AWS]
Interface endpoints	3 endpoints × 2 AZs, per hour and per GB
Classic Load Balancer	Per hour and per GB
CloudWatch Logs	5 control plane log types; 365-day retention, deleted together with the cluster
ECR, S3, DynamoDB	Small: storage and requests

Check the actual spend in AWS Cost Explorer for the account. Tear down (RB-02) when the environment is not in use.

5 Monitoring

5.1 Daily checks

```
kubectl -n argocd get applications           # Synced / Healthy
kubectl -n dev-eks get pods                 # Running, restart counts stable
kubectl -n dev-eks get hpa                  # frontend CPU below target
kubectl get nodes                           # Ready
gh run list --limit 10                      # recent CI results
```

5.2 What to watch in Grafana

The kube-prometheus-stack ships dashboards for the cluster, nodes, namespaces and workloads. Watch:

- Pod restarts and OOMKilled containers in dev-eks (dashboard **Kubernetes / Compute Resources / Namespace (Pods)**).
- CPU and memory per pod against the requests and limits in Appendix A.
- Node CPU and memory: with 2 t3.medium nodes the cluster has little headroom.
- frontend replica count (HPA activity).

There are no alert rules for the application yet; Alertmanager only has the default rules. SLO-based alerts are on the roadmap.

5.3 Logs

- Pod logs: `kubectl -n dev-eks logs deploy/<service>-dev --tail 100` (add `--previous` after a crash).
- Control plane logs (api, audit, authenticator, controllerManager, scheduler): CloudWatch Logs group `/aws/eks/khaipd18-eks-cluster/cluster`.
- CI logs: `gh run view <run-id> --log-failed`.

6 Incident playbooks

6.1 IR-01 CI fails at “Configure AWS credentials”

Symptom	Could not assume role with OIDC: Not authorized to perform sts:AssumeRoleWithWebIdentity or The web identity token provided could not be validated.
Likely cause	AWS_ACCOUNT_ID points to a wrong or retired account; the roles or the OIDC provider do not exist there; the token's sub does not match the trust policy (for example the apply job did not use environment production).
Diagnose	gh variable list; in the account: aws iam get-role --role-name github-actions-ecr-oidc-role --query Role.AssumeRolePolicyDocument; compare the allowed sub values with the job (branch main, pull_request, or environment:production).
Fix	Correct the variable, or run RB-01 step 4 to create the roles. To pause AWS steps entirely, delete the variable: CI then lints, tests, builds and scans only.

6.2 IR-02 Terraform state lock not released

Symptom	Error acquiring the state lock with a lock ID, although no apply is running.
Likely cause	A previous run was cancelled or crashed while holding the DynamoDB lock.
Diagnose	Make sure no Terraform job is running: gh run list --workflow terraform.yaml. Note the lock ID and who holds it from the error message.
Fix	From the workstation with admin credentials: cd terraform && terraform force-unlock <lock-id>. Never unlock while another apply is running: two concurrent applies can corrupt the state. The state bucket is versioned, so a damaged state can be restored from a previous object version.

6.3 IR-03 Terraform Apply does not start

Symptom	The run shows Waiting on the Terraform Apply job.
Likely cause	The production environment needs a reviewer's approval.
Diagnose	gh run view <run-id>; the job is pending a deployment review.
Fix	Read the plan from the pull request, then approve in the run page (Review deployments). Reject if the plan is not what was reviewed.

6.4 IR-04 GitOps write-back rejected

Symptom	Step “Update GitOps and Push Back” fails with GH013: Repository rule violations found / Changes must be made through a pull request. The image is in ECR but the cluster still runs the old tag.
Likely cause	Ruleset protect-main requires pull requests, and a personal repository cannot put the GitHub Actions app on the bypass list.
Diagnose	gh run view <run-id> --log-failed.
Fix	Short term: update the tag in gitops/dev-eks/values-<service>.yaml yourself (pull request or admin push), using the SHA from the run; or disable the ruleset while CI runs (RB-01 warning). Permanent fix (roadmap): deploy key on the bypass list, or let the bot open pull requests.

6.5 IR-05 Argo CD application OutOfSync or Degraded

Symptom	<code>kubectl -n argocd get applications</code> shows OutOfSync, Degraded or Progressing for a long time.
Likely cause	Invalid values (render error), a pod that never becomes ready, a resource changed by hand, or a field managed by another controller.
Diagnose	<code>kubectl -n argocd describe application <service>-dev</code> (conditions and sync result); <code>kubectl -n dev-eks get events --sort-by=.lastTimestamp tail -20</code> ; render locally with <code>helm template</code> .
Fix	Fix the values in Git. For a stuck sync, trigger a refresh in the Argo CD UI (Refresh → Hard refresh) or <code>kubectl -n argocd annotate application <service>-dev argocd.argoproj.io/refresh=hard --overwrite</code> .

6.6 IR-06 Pod in ImagePullBackOff

Symptom	Pod status ImagePullBackOff or ErrImagePull.
Likely cause	The tag does not exist in ECR (CI did not push it, or the values still point to another account); the node cannot reach ECR.
Diagnose	<code>kubectl -n dev-eks describe pod <pod></code> (Events show the image and error); <code>aws ecr describe-images --repository-name <service> --image-ids imageTag=<tag></code> ; confirm the registry account in the image matches the current account.
Fix	Run the service's CI workflow to build and push the tag, or roll back to an existing tag (RB-05). If ECR is unreachable, check the <code>ecr.api/ecr.dkr</code> endpoints and the S3 gateway endpoint route in the private route table.

6.7 IR-07 Pod in CrashLoopBackOff or OOMKilled

Symptom	Restart count rising; <code>kubectl get pods</code> shows CrashLoopBackOff; describe shows Last State: Terminated, Reason: OOMKilled.
Likely cause	Memory limit too low, a bad environment variable, or a dependency that is not reachable at start-up.
Diagnose	<code>kubectl -n dev-eks logs <pod> --previous</code> ; <code>kubectl -n dev-eks describe pod <pod></code> ; compare usage in Grafana with the limits.
Fix	Correct the values (RB-06) or roll back (RB-05). Past example: <code>GOMEMLIMIT: "230Mi"</code> made frontend crash at start-up because Go only accepts units like <code>MiB</code> ; the fix was <code>230MiB</code> (commit feb47a7).

6.8 IR-08 Pod rejected by Pod Security Admission

Symptom	ReplicaSet events show Error creating: pods ... is forbidden: violates PodSecurity "restricted:latest".
Likely cause	The values override the chart's secure defaults (root user, privilege escalation, missing seccomp profile, added capabilities).
Diagnose	<code>kubectl -n dev-eks describe rs -l app=<service>-dev</code> ; the message lists every violation.
Fix	Remove the override from the values file. Do not relax the namespace label; if a workload really needs it, document the exception first.

6.9 IR-09 Calls between services time out

Symptom	A page or checkout fails with gRPC DeadlineExceeded / Unavailable after a change; the target pod is healthy.
Likely cause	The caller is not listed in the target's <code>networkPolicy.allowFrom</code> , or the port changed.
Diagnose	<code>kubectl -n dev-eks get networkpolicy <target>-dev -o yaml</code> ; test from a pod with the caller's label versus another pod.
Fix	Add the caller's release name to the target's <code>allowFrom</code> (RB-06).

6.10 IR-10 Shop not reachable from the internet

Symptom	The shop URL times out, or EXTERNAL-IP stays <pending>.
Likely cause	The load balancer is still provisioning (DNS can take minutes); no healthy frontend pod; the Service was deleted.
Diagnose	<code>kubectl -n dev-eks describe svc frontend-external-dev</code> (events); <code>kubectl -n dev-eks get pods -l app=frontend-dev</code> ; check the instance health in the EC2 console under Load Balancers.
Fix	Wait for provisioning; fix frontend (IR-06/IR-07); let Argo CD recreate the Service (sync frontend-external-dev).

6.11 IR-11 Pods Pending or node NotReady

Symptom	Pods stay Pending with Insufficient cpu/memory, or a node is NotReady.
Likely cause	Requests exceed what 2 t3.medium nodes provide (for example after scaling); a node failed.
Diagnose	<code>kubectl describe pod <pod></code> (scheduler message); <code>kubectl describe node <node></code> (allocated resources, conditions); <code>kubectl top nodes</code> .
Fix	Lower replicas or requests, or raise the node group size (RB-09). A failed node in a managed node group is replaced by EKS; if it is not, cordon and drain it and terminate the instance.

6.12 IR-12 HPA shows <unknown> metrics

Symptom	<code>kubectl -n dev-eks get hpa</code> shows <code>cpu: <unknown>/70%</code> ; frontend does not scale.
Likely cause	Metrics Server is missing or not ready, or the pods have no CPU request.
Diagnose	<code>kubectl -n kube-system get deploy metrics-server</code> ; <code>kubectl top pods -n dev-eks</code> ; <code>aws eks describe-addon --cluster-name khaipd18-eks-cluster --addon-name metrics-server</code> .
Fix	Re-apply Terraform so the metrics-server add-on is installed and healthy; keep <code>resources.requests.cpu</code> set in the values.

6.13 IR-13 Node drain blocked

Symptom	<code>kubectl drain</code> or a node group update loops on <code>Cannot evict pod as it would violate the pod's disruption budget</code> .
Likely cause	Draining would leave frontend with no running pod, which the PodDisruptionBudget forbids. This is the intended protection.
Diagnose	<code>kubectl -n dev-eks get pdb</code> ; check where frontend pods run and whether other nodes have room.
Fix	Make room on another node (scale the node group up) so the evicted pod can be rescheduled, then retry. Do not delete the PDB.

6.14 IR-14 Shopping carts emptied

Symptom	Users report that their carts are empty.
Likely cause	redis-cart restarted; its data is in an emptyDir volume and is lost on restart.
Diagnose	<code>kubect1 -n dev-eks get pod -l app=redis-cart-dev</code> (age, restarts).
Fix	Expected behaviour for this demo. A durable store (ElastiCache, or a PersistentVolume) is needed before real use.

7 Disaster recovery

What is lost	How it is recovered
The whole cluster or account	RB-01 rebuilds everything from Git: infrastructure (Terraform), applications (Argo CD), images (CI). About 45 minutes.
Terraform state	The S3 bucket is versioned: restore the previous object version of dev / terraform.tfstate.
A bad deployment	RB-05 (revert the tag commit).
Cart data	Not recoverable (no persistence).
Grafana settings and Prometheus history	Not backed up; dashboards come back with the chart, history does not.

Appendix A. Service and port reference

Service	Language	Port	Accepts calls from	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi
productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi
shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

frontend-external-dev is a LoadBalancer Service (80 → 8080) that targets the frontend-dev pods.

Appendix B. Command reference

```
# State of everything
kubectl -n argocd get applications
kubectl -n dev-eks get deploy,pods,svc,hpa,pdb,networkpolicy

# One service
kubectl -n dev-eks describe pod -l app=<service>-dev
kubectl -n dev-eks logs deploy/<service>-dev --tail 100 [--previous]
kubectl -n dev-eks rollout status deploy/<service>-dev

# CI
gh run list --limit 10
gh run view <run-id> --log-failed
gh workflow run <language>-services-ci.yaml --ref main

# Render a release locally
helm template <service>-dev helm-charts -f gitops/dev-eks/values-<service>.yaml -n dev-eks

# Terraform (workstation, break-glass only)
cd terraform && terraform init && terraform plan
terraform force-unlock <lock-id>
```

Appendix C. Versions

Component	Version	Defined in
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml